

Perfect—this is the level that actually gets you hired. Below are **50 Amazon/Google-level SQL problems**, focused on **real-world Data Engineering + Analytics scenarios**.

ADVANCED SQL PRACTICE (50 QUESTIONS)

◆ **1. Window Functions & Ranking (10)**

1. Find **top 3 salaries per department**
 2. Find employees with **salary greater than previous employee**
 3. Calculate **running total of salary by department**
 4. Find **difference between current and previous salary**
 5. Rank employees by salary within location
 6. Find **nth highest salary (dynamic n)**
 7. Get employees whose salary is **higher than department average**
 8. Find **first and last hired employee per department**
 9. Calculate **moving average salary (last 3 employees)**
 10. Identify employees whose salary **increased compared to previous record**
-

◆ **2. Joins & Complex Queries (10)**

11. Find employees who **never had a department assigned**
 12. Get departments with **no employees**
 13. Find employees working in **same department & same salary**
 14. Find manager with **maximum team size**
 15. Get employee + manager name (self join)
 16. Find employees who **joined after their manager**
 17. List departments where **avg salary > company avg**
 18. Find employees working in **multiple locations**
 19. Get employees with **unique salary in department**
 20. Find departments where **all employees earn > 50k**
-

◆ 3. Aggregations & Grouping (10)

21. Find **second highest salary per department**
 22. Get department with **lowest avg salary**
 23. Find **total salary per location**
 24. Count employees hired **per year**
 25. Find department with **highest employee count**
 26. Find **median salary per department**
 27. Calculate **salary distribution buckets (0–50k, 50–100k, etc.)**
 28. Find employees contributing to **top 80% salary (Pareto)**
 29. Get cumulative salary % per department
 30. Find departments where **salary variance is high**
-

◆ 4. Subqueries & CTEs (10)

31. Find employees earning more than **overall average salary**
 32. Get employees whose salary is in **top 10%**
 33. Find employees who have **same salary as someone else**
 34. Find employees whose salary is **not in top 3**
 35. Get employees with salary **above department median**
 36. Find employees hired in **same year as max salary employee**
 37. Find employees earning **less than their manager**
 38. Get departments where **max salary < overall avg salary**
 39. Find employees whose salary is **closest to avg salary**
 40. Get employees with **salary greater than 75th percentile**
-

◆ 5. Data Cleaning & Transformation (5)

41. Remove duplicate employees keeping latest record
 42. Convert rows into columns (**pivot question**)
 43. Split full name into first & last name
 44. Handle NULL salaries and replace with avg
 45. Standardize inconsistent department names
-

◆ 6. Real Data Engineering Scenarios (5)

- 46. Design query for **incremental load (new records only)**
 - 47. Identify **late arriving data**
 - 48. Detect **data anomalies (salary jump > 50%)**
 - 49. Find missing dates in **time-series data**
 - 50. Build daily snapshot of employee salary changes
-

💥 TOP 5 MUST-KNOW (WITH SOLUTIONS)

✅ 1. Top 3 Salaries Per Department

```
SELECT *  
FROM (  
    SELECT *,  
        DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC)  
    AS rn  
    FROM employees  
) t  
WHERE rn <= 3;
```

✅ 2. Employees Earning More Than Department Avg

```
SELECT *  
FROM employees e  
WHERE salary > (  
    SELECT AVG(salary)  
    FROM employees  
    WHERE department_id = e.department_id  
)
```

✅ 3. Second Highest Salary Per Department

```
SELECT *  
FROM (  
    SELECT *,  
        DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC)  
    rnk  
    FROM employees  
) t  
WHERE rnk = 2;
```

4. Remove Duplicates (Keep Latest)

```
DELETE FROM employees  
WHERE emp_id NOT IN (  
    SELECT MAX(emp_id)  
    FROM employees  
    GROUP BY name  
);
```

5. Find Missing Dates

```
SELECT d.date  
FROM calendar d  
LEFT JOIN employees e  
ON d.date = e.hire_date  
WHERE e.hire_date IS NULL;
```

How to Use This (Interview Strategy)

 Practice like this:

- Day 1: Window Functions (1–10)
 - Day 2: Joins (11–20)
 - Day 3: Aggregations (21–30)
 - Day 4: Subqueries (31–40)
 - Day 5: Real Scenarios (41–50)
-